DeliverySimulator

DISTRIBUTED DELIVERY SIMULATOR

Adi Shlomo • Dolev Ishay
Ben-Gurion University

Erlang/OTP | Distributed Systems | Fault Tolerant

The Challenge

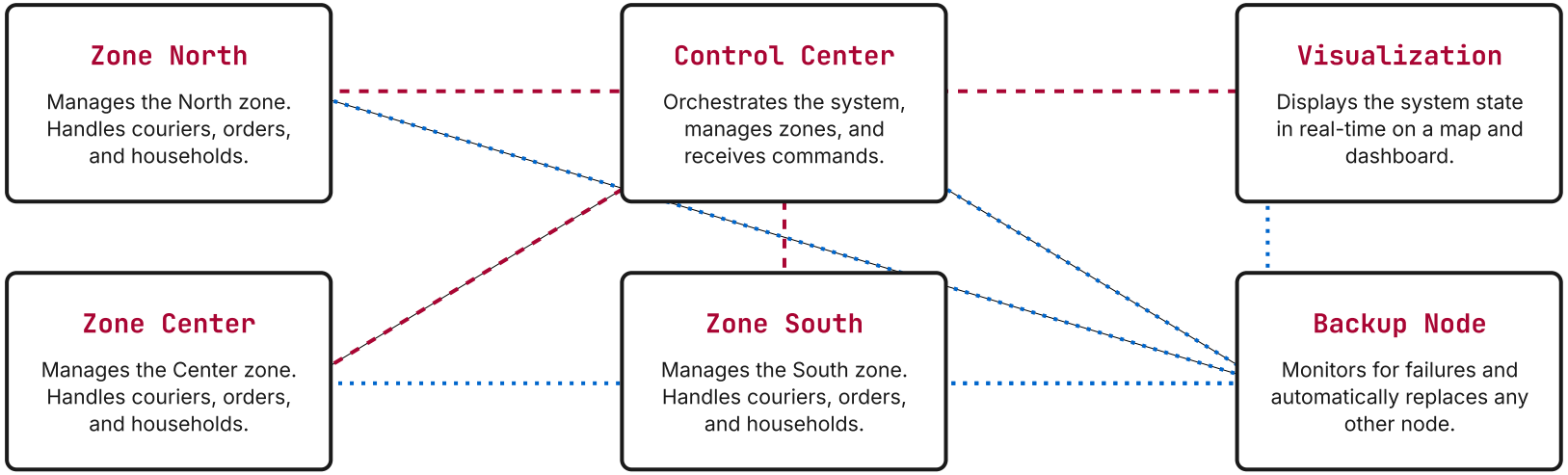
How do modern logistics companies handle **tens of thousands of deliveries** daily?

- << Managing **hundreds of concurrent couriers**
- << Managing **thousands of concurrent houses delivery requests**
- << Optimizing **real-time route allocation**
- << Ensuring **99.999% availability**
- << Handling **node failures gracefully**

Why Erlang/OTP?

- Actor Model
- Supervision Trees
- Hot Code Loading
- Process Isolation
- "Let it Crash"
- Distributed Erlang
- ETS Tables
- Message Passing

Node Structure



6 distributed nodes • 3 geographical zones • 100-1,043 households

Core Components

{gen_server}

- `control_center` - Central coordination
- `households` - Order generation
- `visualization_server` - UI backend

{gen_statem}

- `zone_managers` - Zone orchestration
- `couriers` - Delivery agents
- `packages` - State tracking

State Machine Design

Zone Manager FSM

Courier FSM

Package Lifecycle

Smart Assignment

- << Location-based routing
- << Load balancing
- << Queue management

Real-time Tracking

- << Live position updates
- << Status monitoring
- << Performance metrics

Fault Tolerance in Action

```
%% Backup node monitoring
monitor_loop() ->
  receive
    {nodedown, FailedNode} ->
      take_over(FailedNode),
      %% Recovery in < 5 seconds
    {nodeup, RecoveredNode} ->
      step_down(RecoveredNode)
  end.
```

Erlang/OTP

- << Automatic **failure detection** within 3 seconds
- << State recovery from **ETS snapshots**
- << Seamless **service continuity**

Deployment

```
Erlang/OTP
# Start visualization node
$ rebar3 shell --name visualization@127.0.0.1 --setcookie logistics
> visualization_server:start().

# Start control center
$ rebar3 shell --name control@127.0.0.1 --setcookie logistics
> control_center:start().

# Start zone managers
$ rebar3 shell --name zone_north@127.0.0.1 --setcookie logistics
> zone_manager:start().
```

Supports **distributed deployment** across multiple machines

Concurrency in Action

- << **Household Processes:** Up to 1043 processes, each independently generating orders.
- << **Courier Processes:** Hundreds of processes, each an FSM managing a delivery.
- << **Package Processes:** A separate FSM process for each order, tracking its status.
- << **Zone Manager Processes:** 3 processes concurrently managing all activity in separate zones.
- << **System Processes:** `control_center`, `location_tracker`, `state_collector` and more, all running concurrently.

Control Dashboard

247	1,043	45	3
ACTIVE DELIVERIES	HOUSEHOLDS	COURIERS	ACTIVE ZONES

- << Dynamic courier **addition/removal**
- << Real-time **parameter adjustment**
- << Performance **monitoring**

Performance Metrics

1,000+	<100ms	99.9%	<5s
DELIVERIES/HOUR	RESPONSE TIME	AVAILABILITY	RECOVERY TIME

```
%% Load testing results
100 houses → optimal_performance
200 houses → stable_operation
1043 houses → efficient_load_balancing
```

Erlang/OTP

Technical Achievements

- << Implemented **full distributed system** with 6 autonomous nodes
- << Achieved **automatic failover** with state preservation
- << Built **real-time visualization** with live updates
- << Demonstrated **linear scalability** up to 1,000+ entities
- << Created **hot-swappable** code modules

Leveraging Erlang's **"Let it Crash"** philosophy for robust fault tolerance

Key Insights

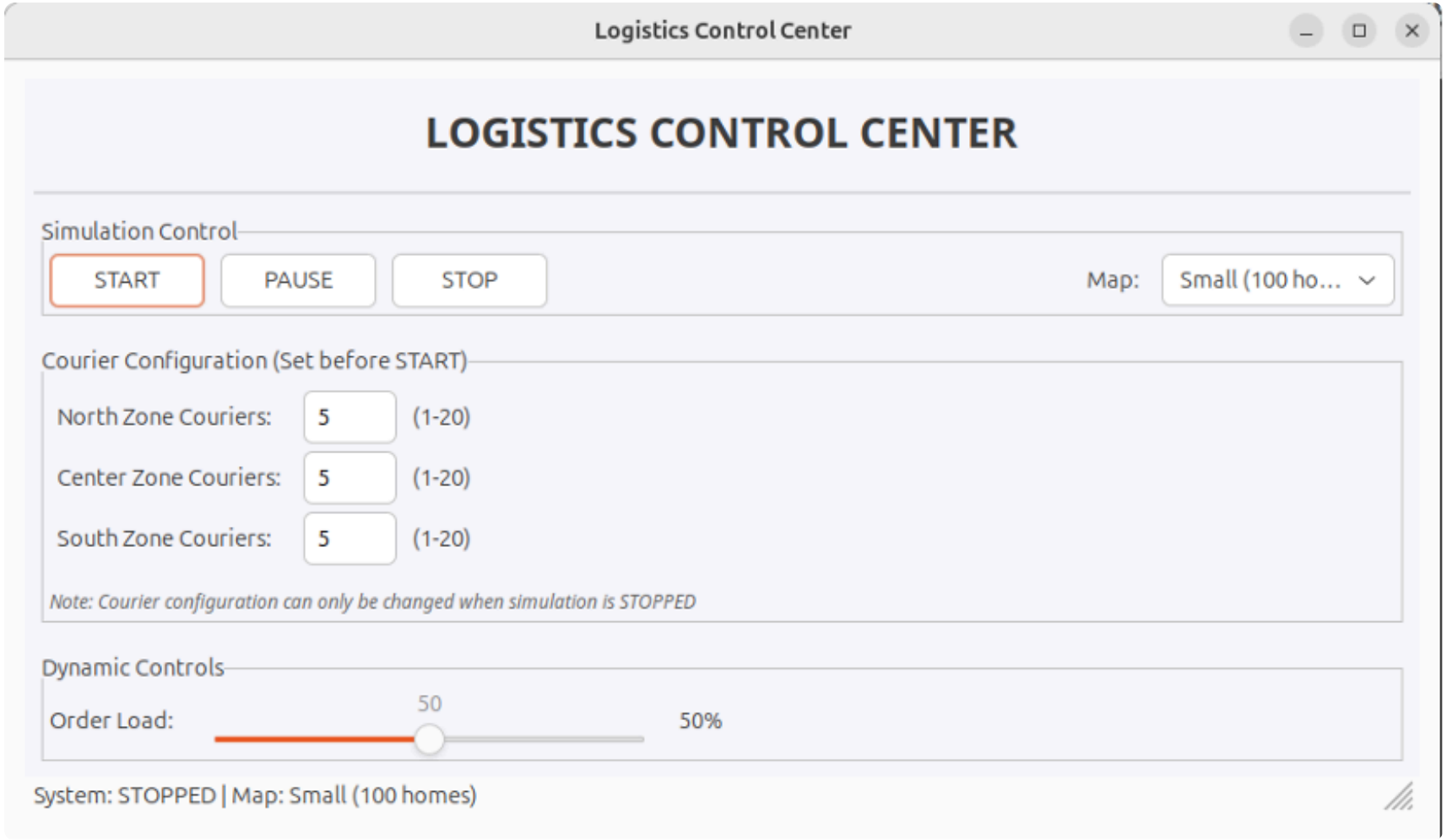
Erlang/OTP Advantages

- << Lightweight processes (millions)
- << Built-in distribution
- << Supervisor hierarchies
- << Pattern matching elegance

Design Patterns

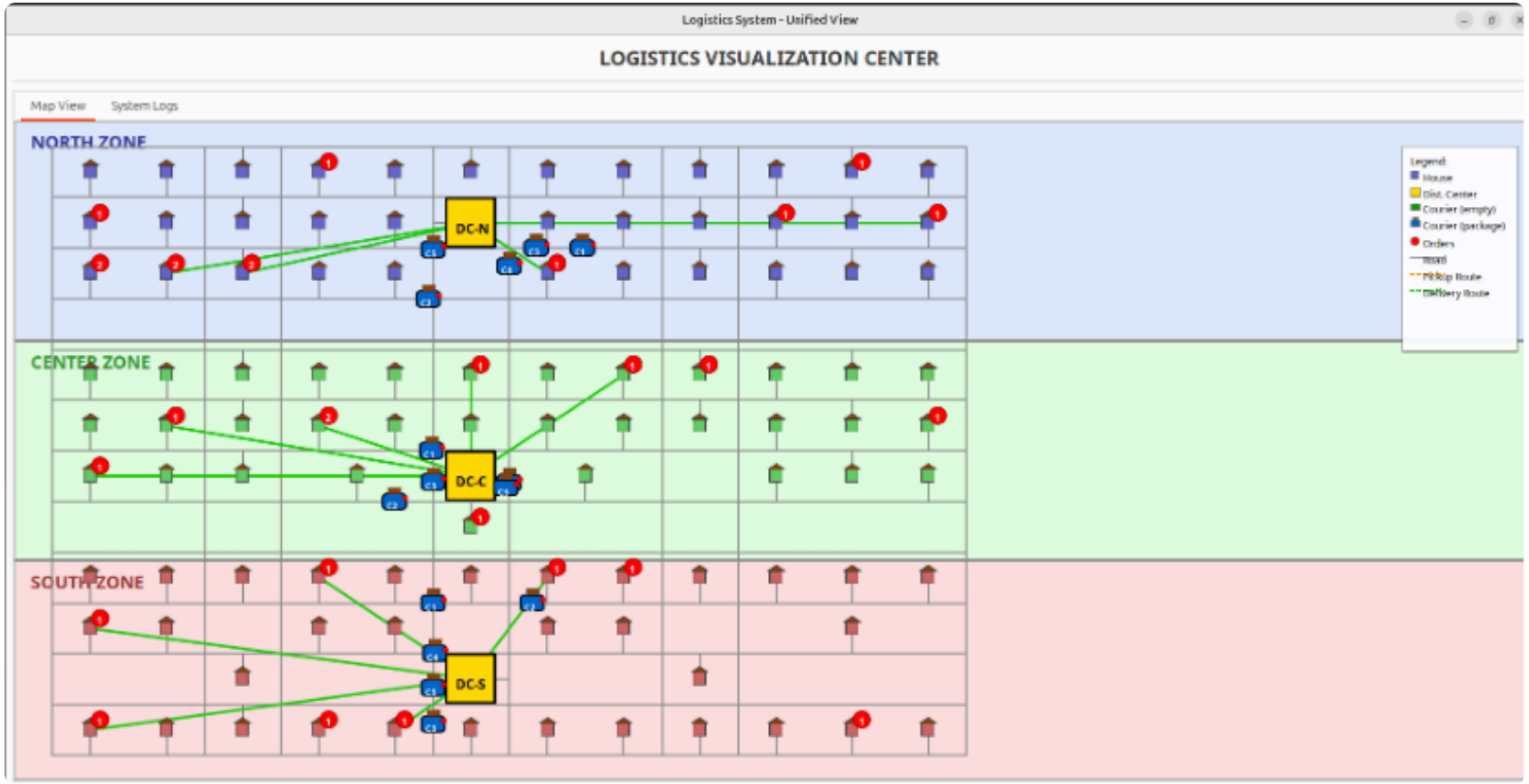
- << Actor model simplicity
- << Message-passing isolation
- << State machine clarity
- << Fault tolerance by design

GUI: Control Center Interface



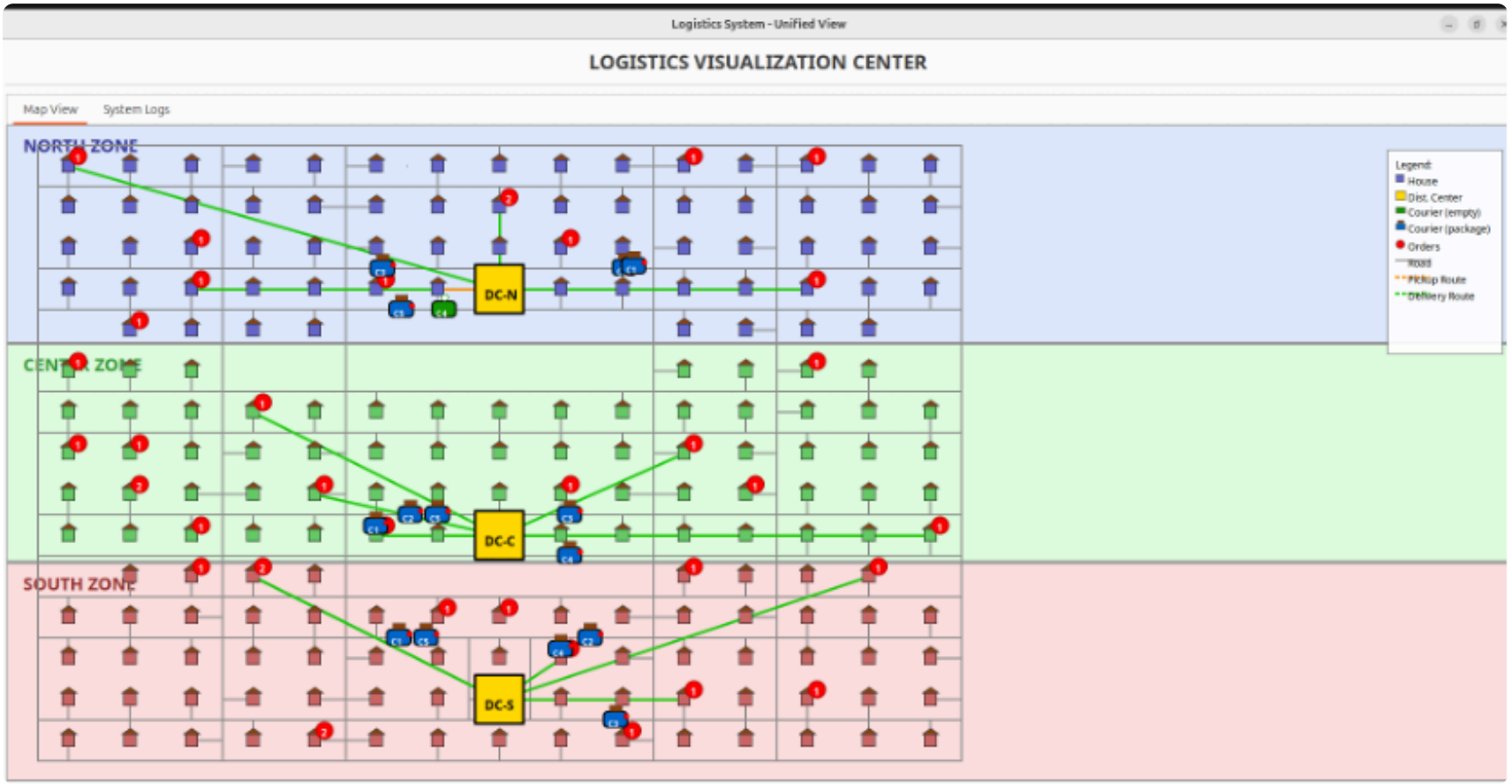
The main dashboard allows for starting, pausing, and stopping the simulation, configuring courier counts per zone, and dynamically adjusting the order load.

GUI: Visualization (100 Households)



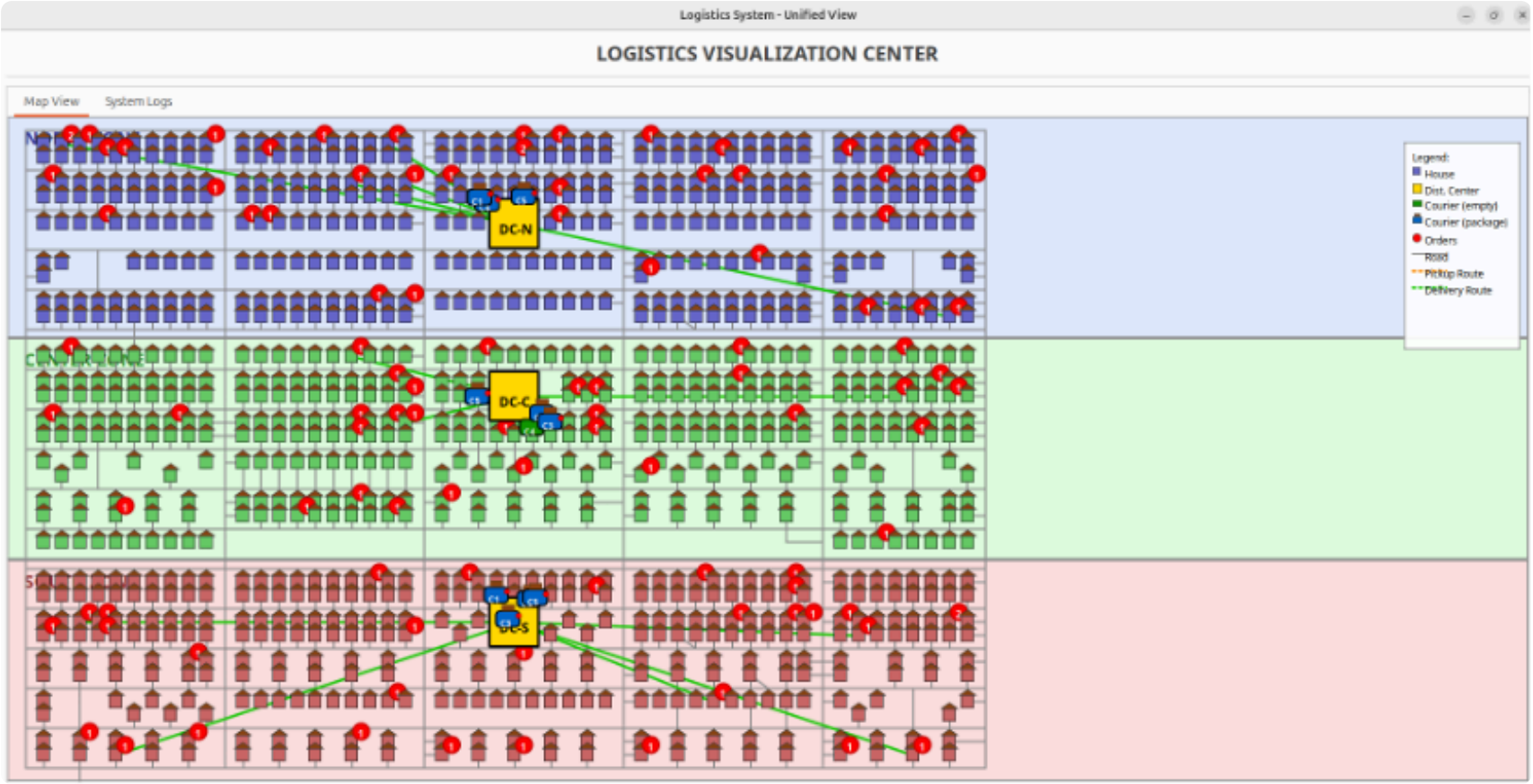
Live map view displaying the simulation with a small load. Couriers, delivery routes, and active orders are tracked in real-time.

GUI: Visualization (200 Households)



The system scaling to a medium load, handling more concurrent deliveries while maintaining real-time updates.

GUI: Visualization (1,043 Households)



Demonstrating high scalability with over a thousand households, showcasing the robustness of the actor model under heavy load.

Thank You

QUESTIONS?

github.com/adi211/erlang-logistics-simulator